

NVIDIA Jetson AGX Orin 64GB Developer Kit

Verificacion, Optimizacion y Preparacion del Sistema

Tutorial Paso a Paso | JetPack 6.2.2 | Marzo 2026

Especificaciones del Sistema Objetivo

Hardware	NVIDIA Jetson AGX Orin Developer Kit 64GB
SoC	Arm Cortex-A78AE 12-core @ 2.2 GHz + Ampere GPU 2048 CUDA cores
AI	275 TOPS (INT8) 64 Tensor Cores 2x NVDLA v2.0
Memoria	64GB LPDDR5 unificada (CPU+GPU) @ 204.8 GB/s
OS	Ubuntu 22.04.5 LTS (Jammy) Kernel 5.15.185-tegra
JetPack	6.2.2+b24 (L4T R36.5.0)
CUDA	12.6 (V12.6.68) cuDNN 9.3.0 TensorRT 10.3.0
OpenCV	4.8.0
Almacenamiento	64GB eMMC 5.1 + M.2 Key M (NVMe SSD)
Potencia	15W - 60W configurable (MAXN hasta 60W)

Contenido

Fase 1	Verificacion del Sistema Comprobar que tu JetPack 6.2.2 esta correctamente instalado
Fase 2	Rendimiento Maximo Modos de potencia, jetson_clocks y optimizacion termica
Fase 3	Memoria y Almacenamiento Swap, ZRAM, NVMe SSD y estructura de directorios
Fase 4	Docker y Contenedores Instalacion de Docker con soporte GPU para Jetson
Fase 5	Herramientas de Monitoreo jtop, tegrastats y diagnostico del sistema
Fase 6	Preparacion del Entorno Python, entornos virtuales y dependencias base
Apendice	Referencia Rapida Resumen de comandos, troubleshooting y fuentes

Fase 1: Verificación del Sistema

Objetivo de esta fase

Antes de realizar cualquier configuración, debemos confirmar que todos los componentes del stack de IA (JetPack, CUDA, cuDNN, TensorRT, OpenCV) están correctamente instalados y coinciden con las versiones esperadas para JetPack 6.2.2.

JetPack 6.2.2 es la versión de producción más reciente para los módulos Jetson Orin.^[1] Incluye Jetson Linux 36.5 con kernel 5.15 y sistema de archivos raíz basado en Ubuntu 22.04. Esta versión corrige un problema de asignación de memoria CUDA introducido en JetPack 6.2.1.

Paso 1.1: Abrir la Terminal

La terminal es la herramienta principal para interactuar con tu Jetson. Para abrirla, presiona simultáneamente las teclas Ctrl + Alt + T. Aparecerá una ventana oscura con un cursor parpadeante. Ahí es donde escribirás todos los comandos de este tutorial.

Consejo para principiantes

Puedes copiar un comando de este documento y pegarlo en la terminal con Ctrl + Shift + V (notar que en la terminal se usa Shift adicional). Para copiar texto desde la terminal, usa Ctrl + Shift + C.

Paso 1.2: Verificar Sistema Operativo y Kernel

Este comando muestra la versión de Ubuntu y del kernel de Linux que está ejecutando tu Jetson:

```
cat /etc/os-release | grep -E "(NAME|VERSION)="  
uname -srm
```

Salida esperada:

```
NAME="Ubuntu"  
VERSION="22.04.5 LTS (Jammy Jellyfish)"  
Linux 5.15.185-tegra aarch64
```

aarch64 significa que es un procesador ARM de 64 bits (la arquitectura del Jetson). tegra en el kernel indica que es un kernel personalizado de NVIDIA para la plataforma Jetson.

Paso 1.3: Verificar Versión de JetPack

JetPack es el paquete maestro de NVIDIA que agrupa CUDA, cuDNN, TensorRT y otras herramientas. Verifica que tengas la versión correcta:

```
apt show nvidia-jetpack 2>/dev/null | grep -E "(Package|Version)"
```

Salida esperada:

```
Package: nvidia-jetpack  
Version: 6.2.2+b24
```

Paso 1.4: Verificar L4T (Linux for Tegra)

L4T es el sistema operativo base adaptado por NVIDIA para Jetson. La versión R36 corresponde a JetPack 6.x:

```
cat /etc/nv_tegra_release
```

Salida esperada:

```
# R36 (release), REVISION: 5.0, GCID: 40478484 ...
```

La línea R36 confirma JetPack 6.x. REVISION: 5.0 corresponde a JetPack 6.2.2 (Jetson Linux 36.5).

Paso 1.5: Verificar CUDA

CUDA es el framework de cómputo paralelo de NVIDIA que permite ejecutar cálculos en la GPU. Es esencial para cualquier carga de trabajo de IA:

```
nvcc --version
```

Salida esperada:

```
nvcc: NVIDIA (R) Cuda compiler driver
Cuda compilation tools, release 12.6, V12.6.68
Build cuda_12.6.r12.6/compiler.34714021_0
```

Si obtienes 'nvcc: command not found'

Ejecuta: `export PATH=/usr/local/cuda/bin:$PATH` y luego añade esa línea al final de tu archivo `~/.bashrc` para que sea permanente:

`echo 'export PATH=/usr/local/cuda/bin:$PATH' >> ~/.bashrc`

Paso 1.6: Verificar cuDNN

cuDNN (CUDA Deep Neural Network library) optimiza las operaciones de redes neuronales en la GPU:

```
dpkg -l libcudnn9 2>/dev/null | grep ^ii | awk '{print $2, $3}'
```

Salida esperada:

```
libcudnn9-cuda-12  9.3.0.75-1+cuda12.6
```

Método alternativo (verificación por cabeceras):

```
cat /usr/include/aarch64-linux-gnu/cudnn_version.h 2>/dev/null \
| grep -E "CUDNN_(MAJOR|MINOR|PATCHLEVEL)" | head -3
```

Salida esperada:

```
#define CUDNN_MAJOR 9
#define CUDNN_MINOR 3
#define CUDNN_PATCHLEVEL 0
```

Paso 1.7: Verificar TensorRT

TensorRT optimiza modelos de IA para inferencia de alto rendimiento en GPUs NVIDIA:

```
dpkg -l | grep -E "^ii.*tensorrt " | awk '{print $2, $3}'
```

Salida esperada:

```
tensorrt 10.3.0.30-1+cuda12.5
```

Paso 1.8: Verificar OpenCV

OpenCV es la biblioteca de visión por computadora. Viene preinstalada con JetPack:

```
pkg-config --modversion opencv4 2>/dev/null || \  
python3 -c "import cv2; print(cv2.__version__)"
```

Salida esperada: 4.8.0

Paso 1.9: Verificar Acceso a la GPU

A diferencia de las GPUs de escritorio, el Jetson tiene una GPU integrada (iGPU). No existe el comando `nvidia-smi` en Jetson. En su lugar, verificamos la GPU así:

```
ls /dev/nvhost-* 2>/dev/null | head -5  
cat /sys/bus/platform/devices/17000000.gpu/power/runtime_status
```

Si ves una lista de dispositivos `/dev/nvhost-*` y el estado muestra `active` o `suspended`, la GPU es accesible. Más adelante instalaremos `jtop` para monitorear la GPU en tiempo real.

Paso 1.10: Verificar Memoria

```
free -h
```

Salida esperada:

total	used	free	shared	buff/cache	available
Mem:	61Gi	7.3Gi	20Gi	...	

El total debe mostrar aproximadamente 62 GB. La diferencia con los 64 GB físicos se debe a que el firmware reserva una porción de la memoria. Recuerda: en el Jetson, CPU y GPU comparten la misma memoria (arquitectura unificada), a diferencia de las GPUs de escritorio que tienen su propia VRAM dedicada.

Paso 1.11: Verificar Almacenamiento

```
df -h /  
lsblk
```

La eMMC (`mmcblk0p1`) debe mostrar ~57 GB utilizables (de los 64 GB totales, descontando particiones del sistema). Si tienes un SSD NVMe instalado, aparecerá como `nvme0n1`.

Paso 1.12: Script Automático de Verificación

Este script ejecuta todas las verificaciones anteriores de una sola vez. Guardalo y ejecutalo cada vez que quieras un informe completo del sistema:

```
cat > ~/verify_system.sh << 'HEREDOC'
#!/bin/bash
set -euv pipefail

echo "====="
echo " Jetson AGX Orin -- System Verification"
echo " $(date)"
echo "====="
echo ""

# Funcion de verificacion
check() {
    local label="$1" cmd="$2" expected="$3"
    local actual
    actual=$(eval "$cmd" 2>/dev/null)
    if echo "$actual" | grep -q "$expected"; then
        echo "[OK] $label: $actual"
    else
        echo "[FAIL] $label: $actual (esperado: $expected)"
    fi
}

check "OS" "grep VERSION_ID /etc/os-release" "22.04"
check "Kernel" "uname -r" "tegra"
check "JetPack" "apt show nvidia-jetpack 2>/dev/null" "6.2"
check "CUDA" "nvcc --version" "12.6"
check "cuDNN" "dpkg -l libcudnn9 2>/dev/null" "9.3"
check "TensorRT" "dpkg -l tensorrt 2>/dev/null" "10.3"
check "OpenCV" "pkg-config --modversion opencv4" "4.8"

echo ""
echo "--- Memoria ---"
free -h | grep -E "^Mem"
echo ""
echo "--- Almacenamiento ---"
df -h / | tail -1
echo ""
echo "--- CPU ---"
echo "Nucleos: $(nproc)"
echo "Arquitectura: $(uname -m)"
echo "====="
HEREDOC

chmod +x ~/verify_system.sh
~/verify_system.sh
```

Resumen de Verificacion Esperada

Componente	Comando Clave	Valor Esperado
Ubuntu	cat /etc/os-release	22.04.5 LTS
Kernel	uname -r	5.15.185-tegra

Componente	Comando Clave	Valor Esperado
JetPack	apt show nvidia-jetpack	6.2.2+b24
L4T	cat /etc/nv_tegra_release	R36 rev 5.0
CUDA	nvcc --version	12.6 (V12.6.68)
cuDNN	dpkg -I libcudnn9	9.3.0
TensorRT	dpkg -I tensorrt	10.3.0
OpenCV	pkg-config --modversion opencv4	4.8.0
Memoria	free -h	~62 GB total

Fase 2: Rendimiento Maximo

Objetivo de esta fase
Configurar tu Jetson AGX Orin para operar a su capacidad maxima: modo MAXN, frecuencias de CPU/GPU al maximo, ventilador optimizado y gestion termica adecuada. Esto desbloquea los 275 TOPS de rendimiento de IA.

Paso 2.1: Entender los Modos de Potencia

Tu Jetson AGX Orin 64GB soporta cuatro modos de potencia preconfigurados, gestionados con la herramienta `nvpmodel`.^[2] Cada modo limita las frecuencias de CPU, GPU, DLA y la cantidad de nucleos activos:

Propiedad	MAXN (ID 0)	15W (ID 1)	30W (ID 2)	50W (ID 3)
Presupuesto	n/a (hasta 60W)	15W	30W	50W
CPUs Activas	12	4	8	12
CPU Max (MHz)	2201.6	1113.6	1728	1497.6
GPU TPC	8	3	4	8
GPU Max (MHz)	1301	420.75	624.75	828.75
DLA Cores	2	2	2	2
DLA Max (MHz)	1600	614.4	1369.6	1369.6
RAM Max (MHz)	3200	2133	3200	3200

Fuente: [NVIDIA Jetson Linux Developer Guide](#)

MAXN (modo 0) es el modo de maximo rendimiento. No tiene limite de potencia fijo y permite hasta 60W. Los 12 nucleos CPU estan activos, la GPU opera a su frecuencia maxima de 1301 MHz con los 8 TPC (Texture Processing Clusters) habilitados, y la memoria funciona a 3200 MHz.

Paso 2.2: Activar Modo MAXN

Primero, verifica en que modo esta actualmente tu Jetson:

```
sudo nvpmodel -q
```

Si no muestra MAXN (modo 0), cambialo:

```
sudo nvpmodel -m 0
```

Reinicio requerido
Si el cambio de modo requiere modificar el `tpc_pg_mask` de la GPU (por ejemplo, pasar de 15W a MAXN), el sistema pedira reiniciar. Escribe YES cuando lo pregunte y espera a que reinicie. Despues del reinicio, verifica con `sudo nvpmodel -q`.

El modo persiste entre reinicios. No necesitas configurarlo cada vez que enciendes el Jetson.

Paso 2.3: Maximizar Frecuencias con jetson_clocks

jetson_clocks fija las frecuencias de CPU, GPU y memoria (EMC) a sus valores máximos estáticos, evitando que el gobernador de frecuencia las reduzca dinámicamente.^[3]

Sigue esta secuencia en orden:

1. Guardar la configuración actual (para poder restaurarla después):
2. Maximizar todas las frecuencias:
3. Verificar los valores aplicados:

```
# Paso 1: Guardar configuración actual
sudo jetson_clocks --store

# Paso 2: Activar frecuencias máximas
sudo jetson_clocks

# Paso 3: Verificar
sudo jetson_clocks --show
```

Al ejecutar --show, deberías ver las frecuencias de cada núcleo CPU en 2201600 (2.2 GHz), GPU en 1300500000 (1.3 GHz), y EMC en 3199000 (3.2 GHz).

Paso 2.4: Control del Ventilador

Para cargas de trabajo intensivas de IA, se recomienda activar el ventilador a máxima velocidad para mantener temperaturas óptimas.^[4]

```
# Activar ventilador al máximo
sudo jetson_clocks --fan

# Para restaurar el comportamiento automático del ventilador:
sudo jetson_clocks --restore
```

Gestión térmica

El SoC Orin tiene un límite térmico de 105 °C. Cuando la temperatura se acerca a ese umbral, el sistema reduce automáticamente las frecuencias (thermal throttling). Con buena refrigeración (ventilador a máxima velocidad), esto rara vez ocurre en condiciones normales de uso.

Paso 2.5: Hacer Permanente la Configuración de Rendimiento

Para que jetson_clocks se active automáticamente cada vez que enciendas tu Jetson, crea un servicio de systemd:

```
sudo bash -c 'cat > /etc/systemd/system/jetson-maxperf.service << EOF
[Unit]
Description=Maximize Jetson Performance
After=nvmodel.service
```

```
[Service]
Type=oneshot
ExecStartPre=/bin/sleep 5
ExecStart=/usr/bin/jetson_clocks
ExecStartPost=/usr/bin/jetson_clocks --fan
RemainAfterExit=yes

[Install]
WantedBy=multi-user.target
EOF'

sudo systemctl daemon-reload
sudo systemctl enable jetson-maxperf.service
sudo systemctl start jetson-maxperf.service

# Verificar que esta activo:
sudo systemctl status jetson-maxperf.service
```

Fase 3: Memoria y Almacenamiento

Objetivo de esta fase

Optimizar el uso de los 64 GB de memoria unificada, configurar swap apropiado y preparar almacenamiento NVMe SSD para modelos de IA y datos de trabajo.

Paso 3.1: Entender la Memoria Unificada

A diferencia de una PC con GPU dedicada (donde la GPU tiene su propia VRAM), en el Jetson AGX Orin la CPU y la GPU comparten los mismos 64 GB de RAM LPDDR5. Esto tiene implicaciones importantes:

- No existe VRAM separada: cuando un modelo de IA usa 'memoria GPU', esta usando la misma RAM que el sistema operativo.
- El ancho de banda de memoria es de 204.8 GB/s, compartido entre CPU y GPU.
- Es crucial no desperdiciar memoria en procesos innecesarios para maximizar lo disponible para IA.
- La GPU no puede usar swap: solo la memoria física está disponible para operaciones de GPU.

Paso 3.2: Configurar ZRAM y Swap

JetPack 6.2.2 configura ZRAM por defecto como área de swap.^[5] ZRAM comprime datos en RAM en lugar de escribirlos a disco, lo cual es rápido pero consume ciclos de CPU. Para tu AGX Orin con 64 GB, ZRAM es generalmente suficiente para la mayoría de cargas:

Verifica la configuración actual de swap:

```
swapon --show
free -h
```

Para cargas de IA muy pesadas (modelos grandes que necesitan más de 60 GB), se recomienda agregar swap en un SSD NVMe:

```
# Crear archivo de swap de 16 GB en SSD NVMe
sudo fallocate -l 16G /mnt/nvme/swapfile
sudo chmod 600 /mnt/nvme/swapfile
sudo mkswap /mnt/nvme/swapfile
sudo swapon /mnt/nvme/swapfile

# Hacer permanente (agregar a fstab)
echo "/mnt/nvme/swapfile none swap sw 0 0" | \
    sudo tee -a /etc/fstab

# Verificar
swapon --show
```

Importante sobre la GPU y el swap

La GPU integrada del Jetson NO puede usar memoria swap. Los drivers de GPU requieren memoria física (RAM real). El swap solo beneficia a programas de espacio de usuario, lo cual libera más RAM física para que la GPU la utilice. Esto es particularmente relevante para modelos grandes de IA.

Paso 3.3: Configurar SSD NVMe

El Developer Kit de AGX Orin tiene una ranura M.2 Key M para SSD NVMe. Se recomienda fuertemente instalar un SSD para:

- Almacenar modelos de IA (que pueden pesar 5-40 GB cada uno).
- Almacenar imágenes de contenedores Docker.
- Swap adicional para cargas de trabajo pesadas.
- Mejor rendimiento de lectura/escritura comparado con la eMMC interna.

Si ya insertaste un SSD NVMe, verifícalo:

```
# Verificar que el sistema detecte el SSD
lsblk

# Debería aparecer algo como:
# nvme0n1      259:0    0   1T  0 disk
#  nvme0n1p1 259:1    0   1T  0 part
```

Si el SSD está nuevo (sin formatear), prepáralo:

```
# Crear partición (CUIDADO: esto borra el SSD)
sudo parted /dev/nvme0n1 --script mklabel gpt
sudo parted /dev/nvme0n1 --script mkpart primary ext4 0% 100%

# Formatear
sudo mkfs.ext4 /dev/nvme0n1p1

# Crear punto de montaje
sudo mkdir -p /mnt/nvme
sudo mount /dev/nvme0n1p1 /mnt/nvme

# Dar permisos a tu usuario
sudo chown $USER:$USER /mnt/nvme

# Hacer permanente (agregar a fstab)
echo "/dev/nvme0n1p1 /mnt/nvme ext4 defaults 0 2" | \
    sudo tee -a /etc/fstab

# Verificar
df -h /mnt/nvme
```

Paso 3.4: Migrar Sistema de Archivos Raíz a SSD (Opcional)

Para máximo rendimiento, puedes migrar todo el sistema de archivos raíz al SSD NVMe.^[6] Esto acelera todas las operaciones de disco del sistema:

```
# Metodo recomendado: usar jetsonhacks root0nNVMe
git clone https://github.com/jetsonhacks/root0nNVMe.git
cd root0nNVMe

# Copiar el sistema de archivos raíz al SSD
./copy-rootfs-ssd.sh
```

```
# Configurar el servicio de arranque desde SSD
./setup-service.sh

# Reiniciar para aplicar
sudo reboot
```

Después del reinicio, verifica con `df -h /` que la raíz ahora está en `nvme0n1p1`.

Paso 3.5: Estructura de Directorios Recomendada

Organiza tu espacio de trabajo para facilitar la gestión de modelos, proyectos y backups:

```
mkdir -p ~/dev/models      # Pesos de modelos (GGUF, safetensors)
mkdir -p ~/dev/projects    # Código y proyectos
mkdir -p ~/dev/containers  # Volúmenes de Docker
mkdir -p ~/dev/datasets    # Datasets de entrenamiento
mkdir -p ~/dev/scripts     # Scripts de utilidad
mkdir -p ~/dev/logs        # Logs de ejecución
```

Si tienes SSD NVMe

Crea symlinks desde tu home a las carpetas pesadas en el SSD:

```
ln -s /mnt/nvme/models ~/dev/models
```

```
ln -s /mnt/nvme/containers ~/dev/containers
```

Así usas el SSD para datos pesados sin cambiar tus rutas habituales.

Fase 4: Docker y Contenedores

Objetivo de esta fase

Instalar y configurar Docker con soporte GPU (nvidia-container-runtime) para ejecutar contenedores de IA optimizados para Jetson (jetson-containers, Ollama, vLLM, etc.).

Paso 4.1: Por que Docker en Jetson

Docker permite ejecutar aplicaciones de IA en entornos aislados sin contaminar tu sistema base. Esto es especialmente importante en Jetson porque:

- Muchas librerías de IA requieren versiones específicas que pueden entrar en conflicto.
- Los contenedores de NVIDIA (jetson-containers) vienen precompilados para ARM64 + CUDA.
- Puedes probar diferentes frameworks sin riesgo de romper tu instalación.
- Facilita la reproducibilidad y el despliegue de proyectos.

Paso 4.2: Verificar si Docker esta Instalado

En JetPack 6 (a diferencia de versiones anteriores), Docker no viene preinstalado si flasheaste desde una máquina host con SDK Manager.^[7] Verifica:

```
docker --version
# Si aparece "command not found", sigue al Paso 4.3
# Si aparece una version, verifica el runtime:
docker info 2>/dev/null | grep -i runtime
```

Paso 4.3: Instalar Docker con Soporte GPU

Usamos los scripts de JetsonHacks, que manejan automáticamente la compatibilidad de versiones y la configuración del runtime de NVIDIA.^[7]

```
# Clonar el repositorio de instalacion
git clone https://github.com/jetsonhacks/install-docker.git
cd install-docker

# Instalar Docker (incluye downgrade a version compatible)
bash ./install_nvidia_docker.sh

# Configurar: agrega tu usuario al grupo docker
# y establece nvidia como runtime por defecto
bash ./configure_nvidia_docker.sh

# IMPORTANTE: Cierra sesion y vuelve a entrar
# (o reinicia) para que los cambios de grupo surtan efecto
sudo reboot
```

Problema conocido: Docker 28.0.0+

Docker version 28.0.0 y posteriores tienen incompatibilidades con el kernel de Jetson 6.2.^[8] Los scripts de JetsonHacks instalan automáticamente Docker 27.5.1 y marcan los paquetes en 'hold' para evitar actualizaciones accidentales. No ejecute apt upgrade de Docker hasta que NVIDIA libere un parche de kernel compatible.

Paso 4.4: Verificar la Instalación de Docker

Después de reiniciar, verifica que todo funcione:

```
# Verificar version
docker --version
# Esperado: Docker version 27.5.1, build ...

# Verificar que el runtime nvidia esta configurado
docker info | grep -i "default runtime"
# Esperado: Default Runtime: nvidia

# Verificar acceso a GPU dentro de un contenedor
docker run --rm --runtime nvidia \
  nvcr.io/nvidia/l4t-base:r36.4.0 \
  cat /proc/driver/nvidia/version

# Test rapido de CUDA en contenedor
docker run --rm --runtime nvidia \
  nvcr.io/nvidia/l4t-base:r36.4.0 \
  nvcc --version
```

Paso 4.5: Mover Almacenamiento de Docker al SSD

Las imágenes de Docker pueden ocupar decenas de GB. Si tienes un SSD NVMe, mueve el directorio de datos de Docker ahí:

```
# Detener Docker
sudo systemctl stop docker

# Crear directorio en el SSD
sudo mkdir -p /mnt/nvme/docker

# Editar configuracion de Docker
sudo nano /etc/docker/daemon.json
```

Asegurate de que el archivo contenga:

```
{
  "runtimes": {
    "nvidia": {
      "path": "nvidia-container-runtime",
      "runtimeArgs": []
    }
  },
  "default-runtime": "nvidia",
  "data-root": "/mnt/nvme/docker"
```

```
}  
  
# Copiar datos existentes  
sudo rsync -aP /var/lib/docker/ /mnt/nvme/docker/  
  
# Reiniciar Docker  
sudo systemctl start docker  
  
# Verificar que usa la nueva ubicacion  
docker info | grep "Docker Root Dir"  
# Esperado: Docker Root Dir: /mnt/nvme/docker
```


Fase 5: Herramientas de Monitoreo

Objetivo de esta fase

Instalar y aprender a usar las herramientas de monitoreo esenciales: jtop para visualización interactiva y tegrastats para monitoreo programático y logs.

Paso 5.1: Instalar jtop

jtop es la herramienta de monitoreo esencial para Jetson. Muestra en tiempo real CPU, GPU, memoria, temperatura, potencia y frecuencias en una interfaz interactiva.^[9]

```
# Instalar pip si no esta presente
sudo apt update
sudo apt install -y python3-pip

# Instalar jetson-stats (incluye jtop)
sudo pip3 install -U jetson-stats

# Reiniciar el servicio
sudo systemctl restart jtop.service

# Ejecutar jtop
jtop
```

Navegación en jtop:

- Usa las teclas numericas 1-7 o las flechas para cambiar entre tabs.
- Tab 1 (GPU): uso de GPU, temperatura, frecuencia.
- Tab 2 (CPU): uso y frecuencia de cada nucleo.
- Tab 3 (RAM): uso de memoria, swap, EMC.
- Tab 4 (ENGINE): estado de DLA, PVA, NVENC/NVDEC.
- Tab 5 (POWER): consumo en watts de cada riel de potencia.
- Tab 6 (CTRL): controlar nvpmode, jetson_clocks y fan.
- Presiona q para salir.

Paso 5.2: Usar tegrastats

tegrastats es una herramienta de línea de comandos incluida con JetPack que muestra estadísticas del sistema en formato texto, ideal para scripting y logs.^[10]

```
# Ejecutar con actualizacion cada segundo
tegrastats --interval 1000
# Presiona Ctrl+C para detener

# Guardar log a archivo
tegrastats --interval 1000 --logfile ~/dev/logs/tegra.log

# 0 usar tee para ver y guardar simultaneamente
```

```
tegrastats --interval 1000 2>&1 | tee ~/dev/logs/tegra.log
```

Interpretacion de la salida de tegrastats:

La salida contiene campos separados por espacios. Los mas importantes son:

Campo	Significado	Ejemplo
RAM X/Y	Memoria usada/total (MB)	RAM 7467/62827MB
CPU [x%@freq]	Uso y frecuencia por nucleo	CPU [45%@2201]
GR3D_FREQ x%	Uso de GPU (porcentaje)	GR3D_FREQ 85%
VDD_GPU_SOC	Potencia del GPU+SoC (mW)	VDD_GPU_SOC 12500mW
VDD_CPU_CV	Potencia del CPU (mW)	VDD_CPU_CV 4200mW
Temp tj@xx.xC	Temperatura junction	tj@52.5C

Paso 5.3: Script de Monitoreo Automatico

Este script crea un log timestamped de tegrastats para analisis posterior:

```
cat > ~/dev/scripts/monitor.sh << 'EOF'
#!/bin/bash
LOGDIR="$HOME/dev/logs"
mkdir -p "$LOGDIR"
LOGFILE="$LOGDIR/tegra_$(date +%Y%m%d_%H%M%S).log"
echo "Iniciando monitoreo... Log: $LOGFILE"
echo "Presiona Ctrl+C para detener"
tegrastats --interval 1000 2>&1 | \
  while IFS= read -r line; do
    echo "$(date +%H:%M:%S) $line"
  done | tee "$LOGFILE"
EOF

chmod +x ~/dev/scripts/monitor.sh
~/dev/scripts/monitor.sh
```

Fase 6: Preparación del Entorno de Desarrollo

Objetivo de esta fase

Configurar Python, entornos virtuales, variables de entorno y dependencias base para desarrollo de IA en tu Jetson AGX Orin.

Paso 6.1: Actualizar el Sistema (Sin Cambiar JetPack)

```
# Actualizar lista de paquetes y paquetes instalados
sudo apt update && sudo apt upgrade -y

# Instalar herramientas esenciales
sudo apt install -y \
    build-essential \
    git \
    cmake \
    pkg-config \
    curl \
    wget \
    htop \
    nano \
    python3-dev \
    python3-pip \
    python3-venv \
    libopenblas-dev \
    libatlas-base-dev
```

No cambies los paquetes de NVIDIA

Al hacer apt upgrade, los paquetes nvidia-l4t-* y nvidia-jetpack se actualizan dentro de la misma versión mayor. Nunca hagas apt dist-upgrade a menos que sea para actualizar a una nueva versión de JetPack siguiendo la guía oficial de NVIDIA.

Paso 6.2: Configurar Variables de Entorno

Agrega las siguientes líneas al final de tu archivo ~/.bashrc para que CUDA y otras herramientas sean accesibles desde cualquier terminal:

```
cat >> ~/.bashrc << 'EOF'

# === CUDA y herramientas de NVIDIA ===
export PATH=/usr/local/cuda/bin:$PATH
export LD_LIBRARY_PATH=/usr/local/cuda/lib64:$LD_LIBRARY_PATH
export CUDA_HOME=/usr/local/cuda

# === Jetson identificación ===
export JETSON_MODEL="AGX_ORIN_64GB"
export JETPACK_VERSION="6.2.2"

# === Alias útiles ===
```

```
alias jtop="jtop"
alias tegra="tegrastats --interval 1000"
alias powermode="sudo nvpmode -q"
alias maxperf="sudo nvpmode -m 0 && sudo jetson_clocks"
alias syscheck="~/verify_system.sh"
EOF

# Aplicar cambios sin reiniciar
source ~/.bashrc
```

Paso 6.3: Crear Entornos Virtuales de Python

Siempre usa entornos virtuales para aislar dependencias de cada proyecto. Esto evita conflictos entre librerías:

```
# Crear un entorno virtual base para IA
python3 -m venv ~/dev/venv/ai-base

# Activar el entorno
source ~/dev/venv/ai-base/bin/activate

# Actualizar pip
pip install --upgrade pip setuptools wheel

# Instalar librerías base comunes
pip install numpy scipy pandas matplotlib

# Para desactivar el entorno:
deactivate
```

Sobre PyTorch en Jetson

No instales PyTorch desde PyPI (`pip install torch`). Las versiones de PyPI son para x86 y no funcionan en ARM64. Usa la versión oficial de NVIDIA para Jetson:

`pip install torch --index-url https://developer.download.nvidia.com/compute/redist/jp/v62`

O usa un contenedor `jetson-containers` que ya incluye PyTorch precompilado.

Paso 6.4: Test Rápido de Validación

Ejecuta este script para confirmar que el entorno está listo para IA:

```
python3 << 'PYEOF'
import sys
print(f"Python: {sys.version}")
print(f"Arquitectura: {sys.platform}")

try:
    import numpy as np
    print(f"NumPy: {np.__version__}")
except ImportError:
    print("NumPy: NO INSTALADO")

try:
```

```
import cv2
print(f"OpenCV: {cv2.__version__}")
except ImportError:
    print("OpenCV: NO INSTALADO")

try:
    import torch
    print(f"PyTorch: {torch.__version__}")
    print(f"CUDA disponible: {torch.cuda.is_available()}")
    if torch.cuda.is_available():
        print(f"GPU: {torch.cuda.get_device_name(0)}")
except ImportError:
    print("PyTorch: NO INSTALADO (usar contenedor)")

print("\n=== Entorno listo ===")
PYEOF
```

Apendice: Referencia Rapida

Comandos Esenciales

Accion	Comando
Verificar sistema	~/verify_system.sh
Ver modo de potencia	sudo nvpmode -q
Activar MAXN	sudo nvpmode -m 0
Maximizar frecuencias	sudo jetson_clocks
Ventilador al maximo	sudo jetson_clocks --fan
Restaurar frecuencias	sudo jetson_clocks --restore
Monitor interactivo	jtop
Monitor CLI	tegrastats --interval 1000
Ver memoria	free -h
Ver almacenamiento	df -h
Ver temperatura	cat /sys/devices/virtual/thermal/thermal_zone*/temp
Docker: ver contenedores	docker ps -a
Docker: espacio usado	docker system df
Docker: limpiar	docker system prune -a

Resolucion de Problemas Frecuentes

Sintoma	Solucion
nvcc: command not found	export PATH=/usr/local/cuda/bin:\$PATH (agregar a ~/.bashrc)
pkg-config: opencv4 not found	sudo apt install libopencv-dev
libcudnn9 no encontrado	sudo apt install libcudnn9-cuda-12
jtop muestra N/A	sudo systemctl restart jtop.service
JetPack muestra version incorrecta	Reflashear con SDK Manager a JetPack 6.2.2
Docker daemon falla al iniciar	Downgrade: bash ./downgrade_docker.sh (v27.5.1)
GPU no detectada en contenedor	Verificar --runtime nvidia y nvidia-container-toolkit
Error de memoria CUDA	Verificar que estas en JetPack 6.2.2 (corrige bug de 6.2.1)

Sintoma	Solucion
Swap no funciona	swapon --show; verificar permisos (chmod 600)
SSD no detectado	lsblk; verificar conexion M.2 Key M

Especificaciones Tecnicas del AGX Orin 64GB

Componente	Especificacion
GPU	NVIDIA Ampere, 2048 CUDA cores, 64 Tensor Cores
CPU	12-core Arm Cortex-A78AE v8.2 @ 2.2 GHz
Rendimiento IA	275 TOPS (INT8)
DLA	2x NVDLA v2.0
Memoria	64GB LPDDR5, 256-bit, 204.8 GB/s
Almacenamiento	64GB eMMC 5.1 + M.2 Key M (NVMe)
Potencia	15W - 60W configurable
Video Decode	1x 8K30, 2x 4K60, 4x 4K30 (H.265)
Video Encode	1x 4K60, 3x 4K30 (H.265)
CSI Camaras	Hasta 6 camaras, 16 lanes MIPI CSI-2
PCIe	1x x16 + 1x x8 + 3x x1 (Gen4)
USB	3x USB 3.2 Gen2, 3x USB 2.0
Conectividad	1x GbE, 1x 10GbE

Fuentes y Referencias

1. NVIDIA JetPack 6.2.2 - <https://jetsonhacks.com/2026/02/06/jetpack-6-2-2-for-jetson-orin/>

2. NVIDIA Developer Guide - <https://docs.nvidia.com/jetson/archives/r35.1/DeveloperGuide/text/SD/PlatformPowerAndPerformance/JetsonOrinNxSeriesAndJetsonAgxOrinSeries.html>

3. RidgeRun Performance Tuning - https://developer.ridgerun.com/wiki/index.php/NVIDIA_Jetson_Orin/JetPack_5.0.2/Performance_Tuning/Maximizing_Performance

4. NVIDIA Developer Forums - <https://forums.developer.nvidia.com/t/how-to-control-the-fan-to-run-all-the-time/213525>

5. NVIDIA Developer Forums - ZRAM - <https://forums.developer.nvidia.com/t/zram-swap-on-jetson-orin-efficient-enough/277490>

6. JetsonHacks - rootOnNVMe - <https://github.com/jetsonhacks/rootOnNVMe>

7. JetsonHacks Docker Setup - <https://jetsonhacks.com/2025/02/24/docker-setup-on-jetpack-6-jetson-orin/>

8. NVIDIA Forum Docker Fix - <https://forums.developer.nvidia.com/t/error-with-nvidia-container-runtime-with-docker-integration-on-agx-orin-with-jp6-2/324558>

9. JetsonHacks jtop Guide - <https://jetsonhacks.com/2023/02/07/jtop-the-ultimate-tool-for-monitoring-nvidia-jetson-devices/>

10. RidgeRun tegrastats - https://developer.ridgerun.com/wiki/index.php/NVIDIA_Jetson_Orin/JetPack_5.0.2/Performance_Tuning/Evaluating_Performance

11. NVIDIA Jetson AGX Orin Specs - <https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-orin/>

12. NVIDIA Jetson AGX Orin Datasheet - <https://openzeka.com/en/wp-content/uploads/2023/04/jetson-agx-orin-64GB-developer-kit-datasheet-web-us.pdf>